Chapter 8 (Scheduling: The Multi-Level Feedback Queue) — Homework (Simulation)

Zhiyuan Pan

All runs use `mlfq.py` in `ostep-homework/cpu-sched-mlfq/`. Every command below can be rerun with `code/chapter8/run.sh`. To make long traces readable, I used a small helper, `code/chapter8/mlfq_tools.py`, that compresses the per-millisecond trace into runs of `start-end job J Qp`, or counts how much CPU time each job got in a time window.

Two details of the simulator that matter for the answers:

- Queues are numbered from the bottom: with `-n 3`, **Q2 is the highest priority** and Q0 the lowest. The allotment (`-a` / `-A`) is counted in time slices.
- Without `-S`, the simulator implements the new **Rule 4**: a job keeps its remaining allotment across I/Os and is demoted once it has used it up. With `-S`, it uses the old **Rules 4a/4b**: issuing an I/O resets the job's allotment, so it stays at the same level.

Q1

> Run a few randomly-generated problems with just two jobs and two queues; compute the MLFQ execution trace for each. Make your life easier by limiting the length of each job and turning off I/Os.

Command: `./mlfq.py -s SEED -n 2 -j 2 -m 20 -M 0`, where `-m 20` limits the job length to 20 ms and `-M 0` turns I/O off. The defaults are a 10 ms quantum and an allotment of 1 slice per queue.

Seed 1: Job 0 runtime 3, Job 1 runtime 15.

time	what runs	why
0–3	Job 0 in Q1	both start in Q1; Job 0 is first and finishes in 3 ms
3–13	Job 1 in Q1	uses its full 10 ms slice, is demoted to Q0
13–18	Job 1 in Q0	finishes its remaining 5 ms

Response: 0, 3 (avg 1.5). Turnaround: 3, 18 (avg 10.5).

Seed 2: Job 0 runtime 19, Job 1 runtime 2.

time	what runs	why
0–10	Job 0 in Q1	uses its full slice, demoted to Q0
10–12	Job 1 in Q1	Q1 has priority over Q0; Job 1 finishes
12–21	Job 0 in Q0	finishes its remaining 9 ms

Response: 0, 10 (avg 5). Turnaround: 21, 12 (avg 16.5). The short job gets ahead of the long one after just one time slice, which is how MLFQ approximates SJF.

Seed 3: Job 0 runtime 5, Job 1 runtime 8. Both jobs fit in one 10 ms slice, so they simply run one after the other in Q1: Job 0 during 0–5, Job 1 during 5–13. Response 0, 5; turnaround 5, 13.

My hand-computed traces matched `-c` in all three cases.

Q2

> How would you run the scheduler to reproduce each of the examples in the chapter?

All examples use three queues (`-n 3`) with a 10 ms time slice (`-q 10`) and an allotment of one slice, unless noted. Jobs are given with `-l start,runtime,ioFreq:...`.

Figure	Command	Result
8.2 Single long-running job	`-n 3 -q 10 -l 0,200,0`	Q2 0–10, Q1 10–20, Q0 20–200
8.3 (left) Along came a short job	`-n 3 -q 10 -l 0,180,0:100,20,0`	A sinks to Q0 by 20; B arrives at 100, runs in Q2 (100–110) and Q1 (110–120), finishes; A resumes 120–200
8.3 (right) Interactive job with I/O	`-n 3 -q 10 -i 5 -S -l 0,175,0:50,25,1`	B runs 1 ms then does a 5 ms I/O; it stays in Q2 the whole time (runs at 50, 56, 62, ...), A runs in Q0 in between
8.4 (left) No priority boost	`-n 3 -q 10 -i 1 -S -l 0,200,0:100,50,1:100,50,1`	B and C alternate in Q2 from 100 to 200; A (in Q0) gets **no CPU at all** during that time: it starves
8.4 (right) Boost every 100 ms	`-n 3 -q 10 -i 1 -S -B 100 -l 0,200,0:100,100,1:100,100,1`	A is boosted to Q2 at 100, 200, 300 and runs 10 ms each time (100–110, 200–210, 300–310)
8.5 (left) Gaming, old Rules 4a/4b	`-n 3 -q 10 -i 1 -S -l 0,300,0:80,200,9`	B runs 9 ms, does a 1 ms I/O, stays in Q2: B gets **90%** of the CPU in [100, 300)
8.5 (right) Gaming tolerance, Rule 4	`-n 3 -q 10 -i 1 -l 0,300,0:80,200,9`	B is demoted to Q1 and then Q0 despite its I/Os; it gets only **31.5%** in [100, 300)
8.6 Lower priority, longer quanta	`-n 3 -Q 10,20,40 -A 2,2,2 -l 0,140,0:0,140,0`	each job runs 2 × 10 ms in Q2, 2 × 20 ms in Q1, then 40 ms slices in Q0

Notes on how the parameters were chosen:

- **Figure 8.3 (right)** describes Rule 4b ("if a job gives up the CPU before the allotment is up, it stays at the same priority level"), so it needs `-S`. Without `-S`, B would be demoted after using up 10 ms of CPU in total.
- **Figure 8.4** needs two interactive jobs that always keep Q2 busy: each runs 1 ms and then does a 1 ms I/O (`-i 1`), so whenever one is doing I/O the other is ready, and A never gets a turn.
- **Figure 8.6**: "20 ms at the highest queue with a 10 ms time slice, 40 ms in the middle with a 20 ms time slice" means an allotment of 2 slices at each level, i.e. `-Q 10,20,40 -A 2,2,2`.

Q3

> How would you configure the scheduler parameters to behave just like a round-robin scheduler?

Use **a single queue**: `-n 1 -q Q`. With only one priority level, Rules 1 and 4 never apply (no job can be higher or lower than another), and Rule 2 (jobs at the same priority run round-robin with the queue's time slice) is all that is left. The quantum `-q` becomes the RR time slice.

Check with jobs of length 100, 200, 300 and a 10 ms slice:

	job 0	**job 1**	**job 2**	**average**
`mlfq.py -n 1 -q 10 -l 0,100,0:0,200,0:0,300,0`	response 0, turnaround 280	10, 490	20, 600	**10, 456.67**
`scheduler.py -p RR -q 10 -l 100,200,300` (chapter 7)	0, 280	10, 490	20, 600	**10, 456.67**

The results are identical. An equivalent alternative is to keep several queues, but give the top queue an allotment so large that no job is ever demoted (e.g. `-n 3 -q 10 -a 1000`), and not use boosting; it gives the same numbers. Without I/O the boost has no effect anyway, because all jobs stay in the top queue.

Q4

> Craft a workload with two jobs and scheduler parameters so that one job takes advantage of the older Rules 4a and 4b (turned on with the `-S` flag) to game the scheduler and obtain 99% of the CPU over a particular time interval.

Command:

```
./mlfq.py -n 3 -q 100 -i 1 -S -l 0,1000,0:0,1000,99 -c
```

- Job 0 is an ordinary CPU-bound job.
- Job 1 is the "cheater": it issues an I/O every 99 ms (`ioFreq 99`), just before its 100 ms time slice runs out. Each I/O takes only 1 ms (`-i 1`).
- With `-S` (old Rules 4a/4b), each I/O resets Job 1's allotment, so it is **never demoted** and stays in the top queue Q2.

Trace (compressed):

```
  0–100  job 0  Q2     job 0 uses its slice and is demoted to Q1
100–199  job 1  Q2     job 1 runs 99 ms, then issues a 1 ms I/O
199–200  job 0  Q1     job 0 gets only the 1 ms while job 1 is doing I/O
200–299  job 1  Q2     job 1 is back in Q2 with a fresh allotment ...
299–300  job 0  Q1
300–399  job 1  Q2     ... and the pattern repeats
```

CPU share over the interval [100, 1100):

	job 0	job 1
with `-S` (old Rules 4a/4b)	10 ms (1.0%)	**990 ms (99.0%)**
without `-S` (new Rule 4)	600 ms (60.0%)	400 ms (40.0%)

With the old rules, Job 1 gets exactly **99%** of the CPU: 99 ms out of every 100 ms. Under the new Rule 4 the same trick no longer works: the scheduler counts all of Job 1's CPU time at a level, so after 100 ms of CPU in Q2 it is demoted, regardless of how many I/Os it issued.

Q5

> Given a system with a quantum length of 10 ms in its highest queue, how often would you have to boost jobs back to the highest priority level (with the `-B` flag) in order to guarantee that a single long-running (and potentially-starving) job gets at least 5% of the CPU?

Answer: boost at least every 200 ms (`-B 200` or smaller).

Reasoning: in the worst case, the long-running job is starved by jobs that keep the top queue busy all the time. The only time it is guaranteed to run is right after a boost. Then it is back in the top queue and gets one quantum (10 ms) before it uses up its allotment and is demoted again. So it gets 10 ms per boost period B:

$$\text{share} = \frac{10\ \text{ms}}{B} \geq 5\% \quad \Rightarrow \quad B \leq \frac{10}{0.05} = 200\ \text{ms}$$

More generally, with quantum q and an allotment of a slices in the top queue, the long job gets a·q per boost period, so B ≤ a·q / 0.05 = 20·a·q.

Simulation: Job 0 is the long CPU-bound job, and Jobs 1 and 2 starve it the same way as in Figure 8.4: each runs 1 ms, then does a 1 ms I/O, with `-S` so they stay in Q2.

```
./mlfq.py -n 3 -q 10 -i 1 -S -B B -l 0,2000,0:0,2000,1:0,2000,1 -c
```

boost period `-B`	CPU used by the long job in [0, 2000)
50	400 ms (20.0%)
100	200 ms (10.0%)
200	**100 ms (5.0%)**
300	70 ms (3.5%)
400	50 ms (2.5%)
no boost	10 ms (0.5%), only its first slice at time 0

The measurements match the formula exactly. With `-B 200` the long job runs at 0–10, 200–210, 400–410, ... — 10 ms out of every 200 ms, exactly 5%.

Q6

> One question that arises in scheduling is which end of a queue to add a job that just finished I/O; the `-I` flag changes this behavior for this scheduling simulator. Play around with some workloads and see if you can see the effect of this flag.

By default, a job that finishes I/O is put at the **back** of its queue. With `-I` it is put at the **front**. In this simulator that means it runs on the very next tick, even ahead of a job at the same level that is in the middle of its time slice.

Workload: one queue, a 10 ms slice, two CPU-bound jobs (100 ms each) and an I/O-bound job that runs 2 ms and then does a 5 ms I/O:

```
./mlfq.py -n 1 -q 10 -i 5 [-I] -l 0,100,0:0,100,0:0,30,2 -c
```

Without `-I`:

```
  0-10   job 0      20-22   job 2      32-42   job 1
 10-20   job 1      22-32   job 0      42-44   job 2   ...
```

Job 2's I/O finishes at 27, but it waits at the back of the queue until both CPU jobs have used a full 10 ms slice. It runs only 2 ms every 22 ms.

With `-I`:

```
  0-10   job 0      22-27   job 0      29-34   job 0
 10-20   job 1      27-29   job 2      34-36   job 2   ...
 20-22   job 2
```

As soon as Job 2's I/O completes, it jumps to the front and runs right away, interrupting Job 0's slice. Job 0 keeps the rest of its slice and continues afterwards.

	job 0 turnaround	**job 1 turnaround**	**job 2 (I/O) turnaround**	**average**
default (back of queue)	208	218	255	227.00
`-I` (front of queue)	220	230	**120**	**190.00**

With `-I`, the I/O-bound job finishes **more than twice as fast** (255 → 120) and the average turnaround improves, while the CPU-bound jobs lose only 12 ms each. A second test with three queues and two I/O jobs (`-n 3 -q 10 -i 3 -l 0,200,0:0,200,0:0,50,2:0,50,5`) showed the same effect: the I/O jobs' turnaround went from 450/240 to 152/139.

Putting jobs that just finished I/O at the front favors interactive, I/O-bound jobs: they get the CPU back immediately, issue their next I/O sooner, and keep the I/O device busy. This is the same idea as

`IO_RUN_IMMEDIATE` in chapter 4. The cost is a little extra delay and more context switches for CPU-bound jobs.